

AI 활용 기술 문서

태엽새 / HEART OF STEEL · github.com/LeeDoik/codename-clockbird

생성형 AI를 "대화가 풍성해지는 장치"가 아니라, 스테이지마다 다른 클리어 조건 그 자체로 썼습니다.

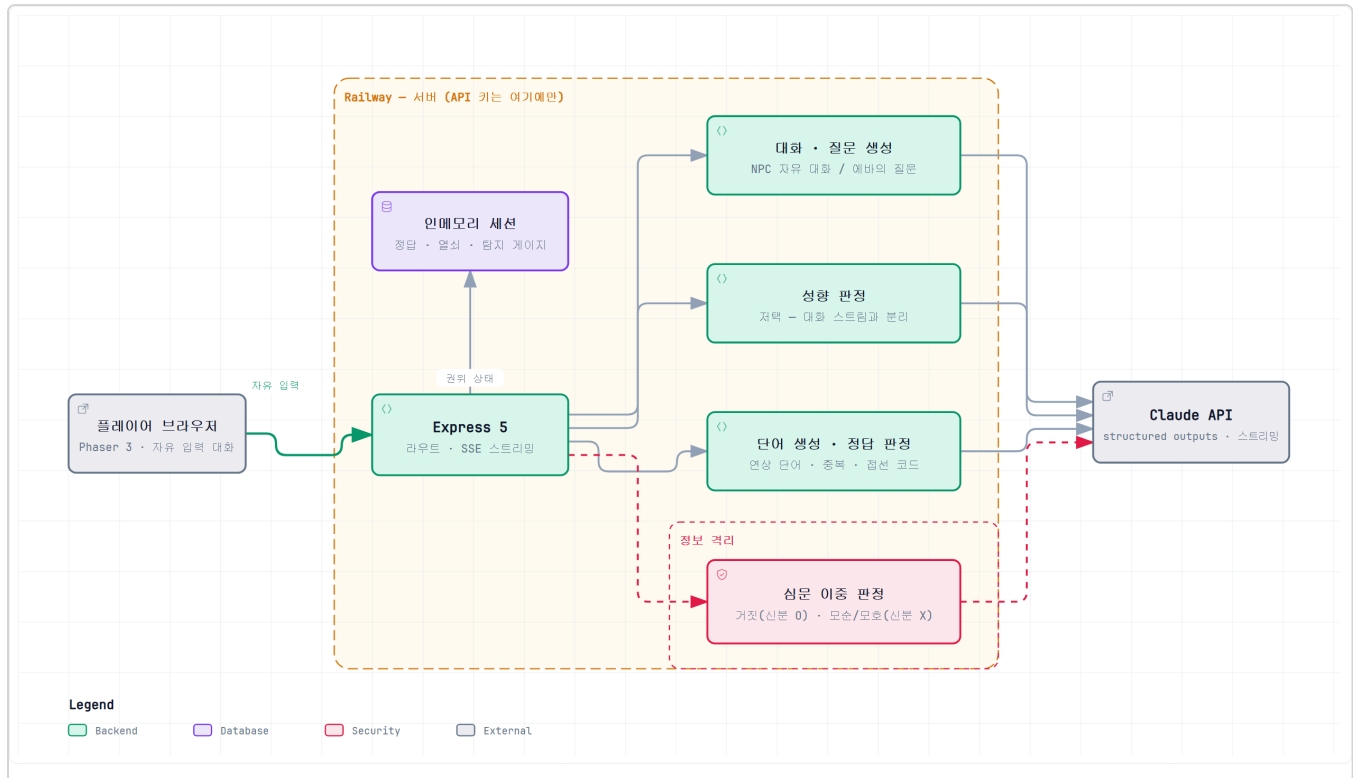
NPC와 잡담이 되는 게임은 많습니다. 이 게임이 택한 방향은 다릅니다 — 네 개 구간의 승패를 가르는 판정을 전부 LLM이 내립니다. 힌트 해석(튜토리얼), 연상 단어 생성과 정답 판정(스테이지 1), 발언의 성향 판정(스테이지 2), 신분 연기에 대한 이중 심문 판정(스테이지 3). AI를 빼면 게임이 성립하지 않습니다.

1. AI 활용 지점 — 7종 호출

#	기능	모델	방식
1	연상 단어 생성 (동료 5인)	claude-sonnet-5	structured outputs · 5인 병렬 · 재시도 + 유출 가드
2	중복·유사어 판정	claude-sonnet-5	LLM-as-judge · 단어 생성 직후 1회
3	접선 코드 정답 판정	claude-sonnet-5	동의어 인정 판정 (튜토리얼·S1 공용)
4	NPC 자유 대화 (본부·거리·저택)	claude-haiku-4-5	페르소나 시스템 프롬프트 + SSE 스트리밍
5	저택 성향 판정	claude-haiku-4-5	대화 스트림과 분리된 별도 호출 · structured
6	심문 질문 생성 (예바)	claude-haiku-4-5	structured · 직전 문답 이력 기반
7	심문 이중 판정 + 신분 추리	claude-sonnet-5	정보 격리된 2개 판정기 병렬 (§3)

모델 이원화 원칙 — 실시간 대화는 저지연·저비용 claude-haiku-4-5, **게임 룰에 영향을 주는 생성·판정**은 claude-sonnet-5. 검문 구현 단계에서 질문 생성을 Haiku로 내려 지연이 절반 이하로 떨어지는 것을 실측하고, 그 분담을 전 스테이지에 적용했습니다. 구현: `src/server/ai/client.js`

2. 아키텍처



구현 위치 — 대화 ai/dialogue.js · 성향 판정 ai/disposition.js · 단어 생성 ai/wordGen.js · 정답·중복 판정 ai/judge.js · 심문 이중 판정 ai/interrogation.js · 유출 검사 ai/guardrail.js(코드 기반, LLM 미사용)

서버 권위. 접선 코드 정답, 열쇠 보유자, 심문의 탐지 게이지는 전부 서버 세션에만 존재합니다. 클라이언트는 결과만 받습니다. 개발 중 `?stage2&key` 플래그가 클라이언트에만 열쇠를 세워 문은 열리는데 서버가 409로 거절한 사고가 있었고, 그 뒤로 **서버 상태를 바꾸는 플래그는 반드시 시작 요청에 실어 보내는 규칙**을 세웠습니다.

structured outputs. 판정·생성 호출은 전부 `anthropic.beta.messages.parse()` + `betaZodOutputFormat(스키마)` 로 JSON 형태를 강제합니다. 파싱 실패 시 재시도하고, 재시도까지 실패하면 게임 로직이 안전한 기본값으로 떨어집니다.

3. 설계 핵심 — 정보 격리

이 프로젝트에서 가장 여러 번 되돌아온 원칙은 하나입니다.

모르는 것은 유출될 수 없다.

가드레일을 출력 필터가 아니라 **정보 설계**로 구현한다.

① 대화 모델은 접선 코드를 아예 모른다

NPC 대화 프롬프트에는 접선 코드를 **넣지 않습니다**. 연상 단어를 만들 때 모델이 쓴 이유(reason)도 넣지 않습니다 — 그 문장은 코드를 알고 쓰인 것이기 때문입니다.

출력 필터로 막지 않은 이유는 스트리밍 때문입니다. 필터가 유출을 감지한 시점에는 **이미 글자가 화면에 찍혀 있습니다**. 필터+재생성은 스트리밍 경로에서 원리적으로 무력하므로, 코드를 반드시 알아야 하는 파이 프라인(연상 단어 생성)에만 적용합니다.

② 심문 로봇 판정기는 플레이어의 신분을 모른다

스테이지 3의 심문은 두 개의 판정기가 같은 답변을 동시에 봅니다.

판정기	받는 정보	판정하는 것
<code>judgeAsSystem</code> (시스템 관점)	신분 단어 포함	답변이 선택한 신분과 명백히 어긋나는가 — 거짓
<code>judgeAsRobot</code> (에바 관점)	신분 단어 미포함 (문답 이력 + 페르소나 만)	앞선 답변과 모순되는가 · 회피하는가 — 모순 / 모호

에바는 플레이어가 무엇을 연기하는지 **모르는 채로** 대화의 앞뒤만 보고 의심합니다. 이 격리가 빠지면 "이 중 AI"는 그냥 호출 두 번이 됩니다. 코드로 확인 가능한 지점입니다 — `src/server/routes/escape.js:161-163` 에서 `judgeAsSystem` 에만 `identityWord` 가 전달됩니다.

③ 모델에게 내부 수치를 주지 않는다

저택 프롬프트에 호감도 0/3 을 넣었더니 모델이 그 숫자를 대사에 그대로 출력했습니다. "적지 마라"로 막는 대신 숫자를 **아예 주지 않고 말로 바꿨습니다**. 같은 원칙의 반복입니다.

4. 주요 프롬프트

모든 프롬프트는 코드가 아니라 **파일**입니다 — `src/data/prompts/*.txt` (10종). 비개발 팀원이 코드를 건드리지 않고 튜닝할 수 있게 하기 위해서입니다(\$5).

연상 단어 생성 — `wordgen-system.txt`

스테이지 1의 퍼즐 전체가 이 프롬프트에서 나옵니다. 핵심은 규칙 1(후보 열거)과 규칙 5(성격이 직접성을 정한다)입니다.

규칙:

1. 먼저 접선 암호가 가지는 특성들을 떠올려라, 색상, 크기, 모양, 용도, 해당 단어만의 특징, 해당 단어와 깊은 연관이 있는 것 등, 모든 요소들 명사 형태로 정리해라.
2. 접선 암호 단어 자체를 쓰지 마라. 그것을 포함한 합성어도 금지다.
3. 접선 암호의 동의어, 외래어 표기, 번역어도 금지다.
(예: 암호가 "툼니바퀴"라면 "기어"도 금지)
5. 선택은, 네 성격·특징이, 규칙 1에서 고른 후보들 중 어느 것을 쓸지와 힌트를 얼마나 직접적으로 줄지를 정한다. (...) 다섯 명이 전부 똑같이 가장 빠른 답 하나로 몰리는 상황만 피하면 된다.
6. 다른 동료가 무엇을 쓸지는 신경 쓰지 마라. 너는 그들의 답을 모른다.

여기서 얻은 교훈. 힌트를 더 직접적으로 만들라고 지시하자 다섯 명의 답이 **한 단어로 몰려 중복률이 83~100%까지 폭등했습니다.** 퍼즐이 성립하지 않는 상태입니다. 규칙 1의 후보 열거를 앞세우고 성격에 따라 직접성이 갈리게 분산을 복원해 최종 42~50%로 내렸습니다(기준 60% 통과). **프롬프트의 한 문장이 게임 밸런스를 통째로 무너뜨릴 수 있다**는 것을 실측으로 확인한 사례입니다. 측정 스크립트:

```
npm run exp:diff
```

심문 이중 판정 — `escape-robot-judge.txt` / `escape-system-judge.txt`

판정 측은 **거짓 · 모순 · 모호** 세 가지입니다. 감점은 합산하지 않고 **가장 무거운 하나만** 반영합니다(모순 100 > 거짓 50 > 모호 25). 결함이 겹칠 때 이중으로 깎이면 플레이어가 이유를 납득하지 못하기 때문입니다.

저택 성향 판정 — `mansion-disposition.txt`

초기 구현은 발언마다 호감도를 +1/-1 씩 세는 숫자 카운터였습니다. 지금은 **홀리스틱 판단**으로 교체되어, 모델이 `act` (지금 확신이 서서 결정적으로 행동) / `wait` (아직 지켜봄) 중 하나를 고릅니다. 대화의 맥락을 보고 판단하므로, 같은 말이라도 앞선 흐름에 따라 다르게 평가됩니다.

5. 프롬프트 운영 — 비개발 팀원용 스튜디오

프롬프트 튜닝이 개발자 병목이 되지 않도록, 브라우저에서 프롬프트를 고치고 즉시 결과를 보는 **프롬프트 스튜디오**(`/prompt-studio`)를 만들었습니다. 10종 템플릿을 목록에서 고르고, 수정본으로 실제 호출을 돌려 응답을 확인한 뒤 반영합니다.

프로덕션에서는 차단됩니다. `NODE_ENV=production`이면 라우트 자체가 등록되지 않고, `/api/studio`는 403을 반환합니다. 배포본에서 실제로 403이 나오는 것을 확인했습니다.

6. 개발에 사용한 AI 도구

도구	용도	활용 내역
Claude Code	코드 작성·리뷰	게임 로직·서버·검증 스크립트 구현, 설계 스펙 문서화, 코드 리뷰. 작업 단위마다 스펙·계획 문서를 남기는 방식으로 진행 (<code>docs/superpowers/</code> 에 스펙 6종·계획 8종)
PixelLab	도트 아트 생성	NPC 스프라이트, UI 요소 50종, 스테이지 3 감시 로봇·에바(방향 별 걷기 포함). 생성물은 <code>scripts/import-*.js</code> 로 규격을 맞춰 반입
Claude (디자인)	타이틀 배경	<code>public/title/title-city.png</code> — 픽셀 아트 증기 도시 2752×1536
Higgsfield	(검토 후 미사용)	초기 캐릭터 생성 검토했으나 크레딧 제약으로 미사용. 해당 시점 스프라이트는 <code>scripts/gen-characters.js</code> 로 절차적 생성

[제출 전 확인] 위 표에 빠진 도구가 있는지 팀원 확인 필요. 특히 오디오 제작에 사용한 도구(있다면)를 여기에 추가해야 합니다.

7. 외부 에셋 · 오픈소스 출처

오픈소스 라이브러리

패키지	라이선스	용도
Phaser 3	MIT	게임 엔진 (렌더링·입력·썬)
Express 5	MIT	HTTP 서버
Vite 7	MIT	클라이언트 번들러
@anthropic-ai/sdk	MIT	Claude API 클라이언트
zod 4	MIT	structured outputs 스키마 정의·검증
dotenv · cross-env	MIT	환경변수 로딩

이미지 · 영상

에셋	출처	비고
NPC 스프라이트 · UI 50종 감시 로봇 · 에바	PixelLab (AI 생성)	팀이 직접 생성·가공. 프롬프트 기록: <code>design/NPC/pixellab-prompts.md</code>
타이틀 배경 <code>title-city.png</code>	Claude 디자인 프로젝트 (AI 생성)	팀이 직접 생성
맵 배경 · 타일	팀 자체 제작 파이프라인	<code>scripts/gen-*-art.js</code> · <code>import-map-art.js</code>
대화창 초상 26종	출처 확인 필요	PixelLab 여부 확인
오프닝 영상 <code>opening.mp4</code>	출처 확인 필요	

오디오

[제출 전 반드시 채울 것 — 미기재 시 심사 제외 사유]

BGM 7종(title · tutorial · stage1 · stage2 · stage3 · minigame1 · minigame2)과 SFX 12종(walk · select · close · gear · steam · switch · boom · clear · fail · warning · text-next · energy-charge)의 **출처와 라이선스**를 기재해야 합니다.

git 기록상 public/audio는 **ilsimdotcom-ai** 계정에서 커밋되었습니다.

- AI 생성이면 → 도구명과 생성 방식
- 무료 에셋이면 → 사이트명 · 원저작자 · 라이선스(CC0 / CC-BY 등) · 원본 링크
- 직접 제작이면 → "팀 자체 제작"으로 명시

8. 검증

AI 파이프라인은 실호출 스모크와 결정론적 정적 검사로 나누어 검증합니다.

명령	검증 내용
<code>npm run smoke</code>	대화 SSE 스트리밍 (실제 LLM 호출)
<code>npm run smoke:tutorial</code>	튜토리얼 대화·정답 판정
<code>npm run smoke:mansion</code>	저택 성향 판정 · 소프트락 회귀 검사
<code>npm run smoke:escape</code>	심문 8종 시나리오
<code>npm run exp:diff</code>	연상 단어 중복률 실측 (밸런스 지표)
<code>npm run check:escape</code> · <code>check:jail</code>	시야·스폰 안전·게이지 산술 (LLM 미사용, 결정론적)